

Day 13: Monitoring, logging, and visualization

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

October 3, 2026

- ▶ On Days 8 to 12 you built applications that detect objects, stream video, and send results with MQTT. A deployed application runs for weeks with no person in front of it.
- ▶ A deployed model can fail with no error message. The light changes, a new object appears, a sensor gets dirty, or the board gets hot. The program still runs, but its answers get worse.
- ▶ Monitoring makes such failures visible. The device measures itself, writes logs that fit its storage, and sends metrics to a dashboard. An alert tells a person before the users see the problem.
- ▶ In the lab you add measurements and logs to the Day 8 detector, show them on a live dashboard, cause a drift event, detect it from the confidence values, and test one alert rule.

After this day you can:

- ▶ Select metrics for model health and for system health.
- ▶ Write structured logs on a device with small storage.
- ▶ Build a live dashboard for an edge device.
- ▶ Detect data drift without labels.

Part 1 Failures of deployed edge AI

Exercise: for three failures, name the metric that shows the failure first

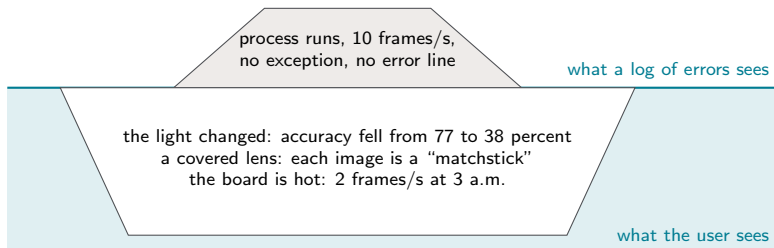
Part 2 Logging

Exercise: calculate the storage that one day of logs needs at three sampling rates

Part 3 Visualization and alerts

Exercise: set alert thresholds from a given metric history

A deployed model fails with no error



- ▶ A program crashes when the code is wrong. A model gives a wrong answer and the program continues.
- ▶ The service is "up" and the answers are wrong. Nobody gets an alert, because no check fails.
- ▶ On a device, nobody watches the screen. The failure stays until a user complains, or until somebody looks at the data.

Key point: Available is not correct. Monitoring must measure the answers, not only the process.

Source: Accuracy and class: experiment of this part.
The frame rates are example values.

Failure	Cause on the device	What the user sees
Silent accuracy loss	The input differs from the training data, or the pre-processing differs	Wrong classes, missed objects
Data drift	Light, season, new objects, a new camera	Slow loss over weeks
Sensor fault	Dirt, a loose cable, a broken or saturated sensor	Strange values, a frozen image
Heat	Bad cooling, a warm room, a dusty fan	Slow answers, a low frame rate
Memory leak	The program keeps data that it never frees	A crash or a restart after hours or days
Storage, clock, network	Full disk, wrong time, lost link	Lost logs, wrong time stamps, lost messages

- ▶ The first two change the answers of the model. The others change the device that runs the model.
- ▶ Most of them do not stop the program. They make it slower or wrong.

Why a wrong answer gives no error

```
probs = softmax(model(x))      # always 1000 values with a sum of 1
label = names[probs.argmax()]  # always one class
print(label, probs.max())      # black image: matchstick 0.026
```

- ▶ A classifier always gives a class. The softmax always gives values with a sum of 1, also for an image of nothing.
- ▶ An exception comes only from a wrong shape, a wrong type, or a missing file. A wrong image has the correct shape.
- ▶ In the experiment, a black image (a covered lens) gives the class “matchstick”. The program prints it and continues.
- ▶ The confidence of 0.026 is a hint. Nobody sees it, unless the program records it.

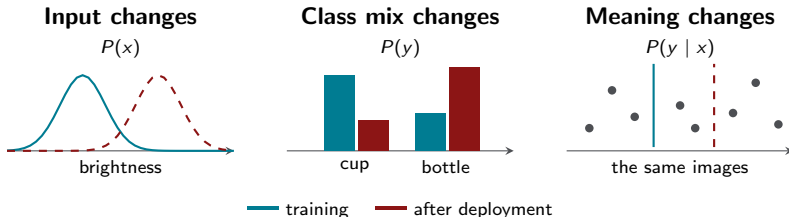
Key point: The model cannot tell you that its input is bad. Your code must check the input and record the output.

Source: Experiment of this part: MobileNetV2 (ImageNet weights), an image of zeros.

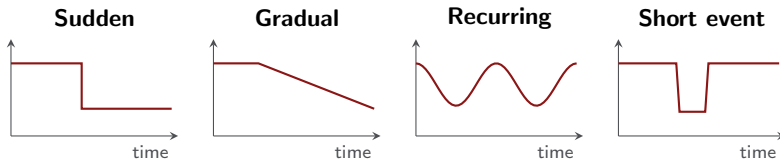
Pre-processing	Accuracy	Mean conf.
Correct	77.4 %	0.770
B, G, R order	67.4 %	0.653
No normalization	46.0 %	0.531
Resize with no crop	76.2 %	0.755

- ▶ The same model file, the same 500 images. Only the code before the model changes.
 - ▶ OpenCV reads images in the order B, G, R. A model of torchvision needs R, G, B.
 - ▶ Each version runs with no error.
-
- ▶ Training-serving skew: the device makes the input in a different way from the training code. The model sees data that it never saw.
 - ▶ Prevention: one pre-processing function for the two sides, and a test with a stored input and its stored output (as the test window of Day 3).

Source: Experiment of this part: MobileNetV2, 500 Imagenette images, PyTorch on an x86 processor.



- ▶ **Input drift ($P(x)$):** the images change, the classes keep their meaning. Light, season, a new camera, dust.
- ▶ **Prior drift ($P(y)$):** the classes come in a new mix. More bottles than cups after a change of the menu.
- ▶ **Concept drift ($P(y | x)$):** the correct label of the same input changes. A new rule says that a cup with a lid is a "bottle". Only new labels show it.

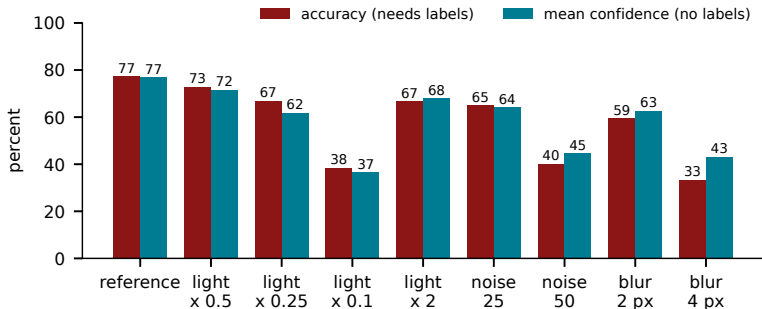


Accuracy (or confidence) over time

Form	Example for a camera	Response
Sudden	Somebody moves the camera	Alert at once, check the device
Gradual	Dust on the lens over weeks	Trend over days, clean or retrain
Recurring	Day and night, seasons	Compare with the same hour of an earlier day
Short event	A person stands in front of the lens	Ignore it, unless it repeats

The form tells you the time window of the check: minutes for a sudden change, days for a gradual one.

Measured: light, noise, and blur



- ▶ Light: each pixel value times a factor ($\times 0.1$: a dark room). Noise: a standard deviation of 25 or 50 (of 255). Blur: a lens out of focus.
- ▶ Each change lowers the accuracy. No change gives an error.
- ▶ The mean confidence falls with the accuracy. With a blur of 4 pixels it falls less: 33 percent correct, a mean confidence of 43 percent.

Source: Experiment of this part: MobileNetV2, 500 Imagenette images.

The input shows which change happened

Condition	Accuracy	Mean confidence	Brightness	Sharpness
Reference	77.4 %	0.770	113.5	1013
Light $\times$ 0.25	67.0 %	0.619	28.4	65
Light $\times$ 0.1	38.4 %	0.365	11.4	12
Light $\times$ 2	66.8 %	0.680	177.6	1821
Noise 50	40.2 %	0.446	114.6	3850
Blur 4 pixels	33.4 %	0.432	113.5	3

- ▶ Brightness: the mean pixel value (0 to 255). Sharpness: the variance of the Laplacian of the grey image. Each needs a few lines of `numpy`.
- ▶ The confidence says that something is wrong. The input statistics say what: dark, too bright, noisy, or not sharp.
- ▶ Darkness also lowers the sharpness, and noise raises it. Read the two statistics together.

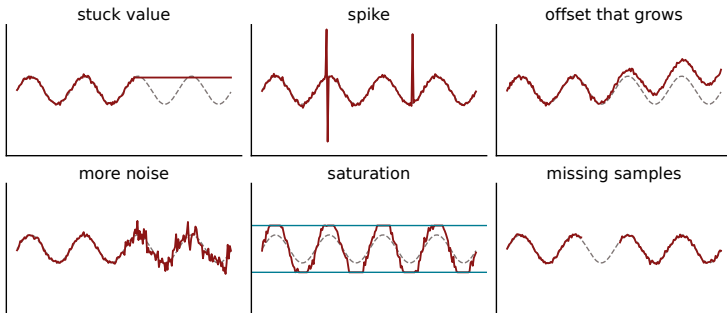
Source: Experiment of this part: means of the 500 images after the centre crop of 224×224 pixels.

The confidence is a signal, not an accuracy

Group	Images	Correct
Confidence ≥ 0.5	387	88.6 %
Confidence < 0.5	113	38.9 %
All	500	77.4 %

- ▶ An image with a low confidence is wrong more often.
- ▶ So a rise of the part with a low confidence is a good sign of trouble.
- ▶ The largest softmax probability is a simple baseline to find wrong and unknown inputs. Better methods exist, but this one needs no change of the model.
- ▶ A model can be sure and wrong: a new kind of object can get a high confidence. A confidence of 0.9 is not “90 percent correct”.
- ▶ Use the confidence as a trend against a reference period. Do not read it as the accuracy.

Source: Experiment of this part (table). The baseline: Hendrycks and Gimpel, 2017 (see the further reading).



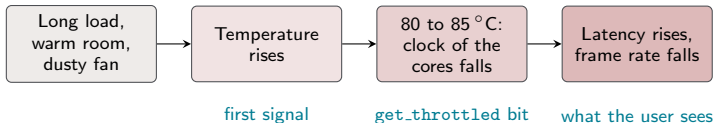
- ▶ Dashed line: the correct signal. Red line: what the sensor gives.
- ▶ A sensor fault changes the input like drift. A repair or a new calibration fixes it, not a new training.
- ▶ Causes: a loose cable (stuck, missing), electrical noise (spike, noise), temperature and age (offset), a wrong range (saturation).

Source: Synthetic signals: no sensor made them.

Fault	Check on the device	Example
Stuck value	The same value for N samples	IMU: variance 0 for 1 s
Spike	Change between two samples above a limit	More than the physics permits
Offset	Mean at rest against the calibration	Accelerometer at rest: 1 g
More noise	Standard deviation at rest above a limit	Twice the value at installation
Saturation	Count of values at the end of the range	16 g of the Day 2 setting
Missing samples	Gaps of the sequence number or of the time	The seq of Day 12
Camera	Black, frozen, or not sharp	Brightness below 5; the same frame twice; low sharpness

- ▶ Each check is a few lines of code and runs before the model.
- ▶ Count the failed checks and send the counts as metrics. Do not send each bad sample.

Heat: from temperature to latency



- ▶ Raspberry Pi: between 80 and 85 °C the firmware lowers the clock of the Arm cores in steps. At 85 °C it also lowers the GPU clock.
- ▶ The cooler of the Raspberry Pi 5 starts at 50 °C and runs at full speed from 75 °C. A stopped cooler shows as a higher temperature at the same load.
- ▶ Day 9: a short run gives a latency that a long run does not keep. A device that runs for days meets the heat.

Key point: The temperature rises minutes before the latency. Record the temperature.

Source: Raspberry Pi documentation, “Frequency management and thermal control” and “Cooling”.

Read the heat state of a Raspberry Pi

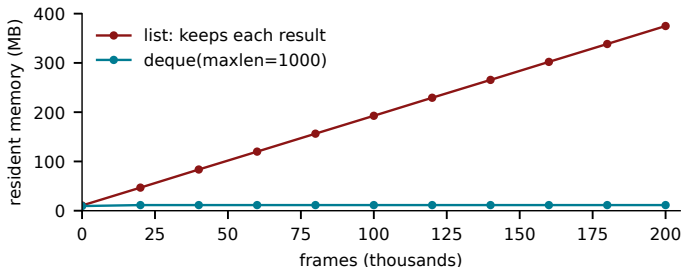
```
cat /sys/class/thermal/thermal_zone0/temp # millidegrees, no root needed
vcgencmd measure_temp                    # temp=... 'C
vcgencmd measure_clock arm                # frequency(0)=... (Hz)
vcgencmd get_throttled                    # throttled=0x0 when all is well
```

Bit	Now	Since the start
Undervoltage	0x1	0x10000
Arm frequency capped	0x2	0x20000
Throttled	0x4	0x40000
Soft temperature limit	0x8	0x80000

- ▶ The “since the start” bits stay set. One read each minute catches a short event.
- ▶ Undervoltage (a weak power supply) also makes a board slow or unstable. The same command shows it.

Source: Raspberry Pi documentation, “vcgencmd”. The temperature function of the Day 8 lab reads the first file.

Memory leak: slow growth



- ▶ One result for each frame: a time, a frame number, 3 detections. One version appends each result to a list. The other keeps the last 1000 results.
- ▶ The list grows by 1910 bytes for each frame. The bounded version stays at 11.4 MB.
- ▶ Other causes: a cache with no limit, objects that a callback keeps, a native library that does not free its memory.

Source: Experiment of this part: resident memory of a Python process, each 20 000 frames.

Growth for each frame (measured)	1910 bytes
At 10 frames each second, each hour	65.6 MB
In a test of 10 minutes	10.9 MB
Time to fill 4 GB (half of the RAM of the board)	62.5 hours

- ▶ A test of 10 minutes shows 10.9 MB. Nobody sees it as a problem.
- ▶ After 2.6 days the system swaps or the kernel stops the process. The application restarts and the leak starts again.
- ▶ Signals: the slope of the resident memory over hours, the number of restarts, and the messages “Out of memory” of the kernel log (`journalctl -k`).

Key point: A leak shows only over a long time. Record the memory of the process each minute and look at the slope.

Source: Experiment of this part (growth for each frame). The other values are calculations; 1 MB = 1024 KB.

Failure	Effect	Signal
Disk full	Logs and the message queue stop; the program can fail	Free space of the disk (Part 2)
Wrong clock	Wrong time stamps, wrong order of events	Offset to an NTP server (Day 12)
Lost link	Messages wait in the queue	Queue length, age of the oldest message (Day 12)
Weak power supply	Slow board, resets	get_throttled bit 0x1
Camera lost	No frames, a frame rate of 0	Count of failed reads
microSD card wear	Read errors, a damaged file system	Errors in the kernel log

- ▶ Many of these come from the device, not from the model. A model metric does not show them.
- ▶ A restart hides most of them for a while. Count the restarts.

Metric	Where to read it	It shows
Latency (p50, p95)	Time of each step in the program	A slow model, heat, a busy CPU
Frame rate, dropped frames	Counters of the program	A slow pipeline, a lost camera
CPU use	<code>psutil.cpu_percent()</code>	Other programs, a busy loop
Temperature, throttled	<code>/sys/class/thermal</code> , <code>vcgencmd</code>	Heat, a bad power supply
Memory of the process	Resident memory (RSS) from <code>psutil</code>	A leak
Free disk	<code>shutil.disk_usage("/")</code>	Logs with no limit
Uptime, restarts	Start time of the program	Crashes

- ▶ System metrics answer: does the device work, and is it fast enough?
- ▶ They are cheap: one read each second or each minute.

Metric	How	It shows
Confidence	Mean, and the part below a threshold, each minute	Drift, a bad input
Class counts	Detections of each class each minute	Prior drift, a missed class
Detections in each frame	Mean and the part with none	A covered or moved camera
Input statistics	Brightness, sharpness, value range	Light, blur, sensor faults
Rejected inputs	Count of failed input checks	Sensor faults

- ▶ Model metrics answer: does the model see what it saw in training, and does it answer as before?
- ▶ Each one needs a reference: the values of a good period after the installation. Part 3 compares with it.
- ▶ The accuracy is the metric that you want, and it needs labels.

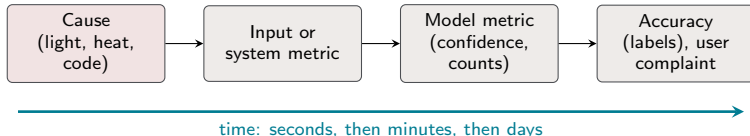
Drop to see	Images in each period
10 points	313
5 points	1180
2 points	7075

Labelled images	95 % interval
50	± 11.6 points
100	± 8.2 points
500	± 3.7 points

- ▶ On a device, nobody labels the images. A person must look at a sample of them.
- ▶ To see a drop of 5 points from 77.4 percent, a person labels 1180 images before and 1180 images after.
- ▶ So the labelled check is slow and expensive. It confirms a problem; it does not find it.
- ▶ The fast check uses the metrics with no labels.

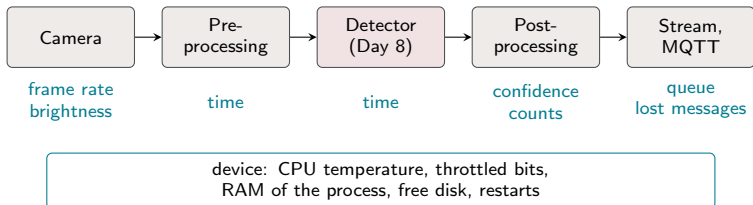
Source: Calculation: test of two proportions (alpha 0.05, power 0.8) from 77.4 %; interval with the normal approximation.

Which metric shows a failure first



- ▶ Measure as close to the cause as you can. The input statistics change with the first dark frame. The confidence needs many frames for a clear mean. Labels need days.
- ▶ A metric close to the cause also says where to look: the camera, the cooler, or the code.
- ▶ The later metrics confirm the effect on the user. Keep them, but do not wait for them.

The metrics of the lab application



- ▶ The Day 8 application already reports the frame rate and the time of each step on its page `/stats`.
- ▶ In the lab you add the temperature, the RAM, and the confidence, write them to a log (Part 2), and show them on a dashboard (Part 3).

Exercise: Name the first metric

An application counts cups with the Day 8 detector on a Raspberry Pi 5. For each failure, name the metric that shows it first, and the metric that the user sees later.

- A. In the evening, the room light is dimmed to one tenth.
- B. The cooler of the board stops in a warm room.
- C. A new version of the program keeps every result in a list for a “history” page.

For each one, also say: which threshold or time window would you use for the check?

- A. First: the mean brightness (11 in place of 113 in the experiment). Then the mean confidence and the detections in each frame fall. Later: missed cups, seen with labels. Check: brightness below a limit for some minutes; compare with the same hour of an earlier day.
- B. First: the temperature rises above its normal value at the same load. At 80 °C the throttled bit is set and the clock falls. Later: a higher latency and a lower frame rate. Check: temperature above about 75 °C for 5 minutes.
- C. First: the resident memory grows in a straight line (about 66 MB each hour at 10 frames each second). Later: swap, a stopped process, a restart. Check: the slope over some hours, and the restarts.

Key point: The first metric is close to the cause: the input, the device, or the process. The accuracy comes last.

Part 1 Failures of deployed edge AI

Exercise: for three failures, name the metric that shows the failure first

Part 2 Logging

Exercise: calculate the storage that one day of logs needs at three sampling rates

Part 3 Visualization and alerts

Exercise: set alert thresholds from a given metric history

Channel	Question	On the edge device
Metrics	Which value left its normal range?	Latency, frame rate, temperature, confidence: numbers at fixed times
Logs	Which part did what, and when?	One record for each event: a start, an error, a slow frame
Traces	Where did one input spend its time?	The time of each step for one frame: capture, model, send
Prediction logs	Did the model behave as before?	A sample of inputs and outputs, for later labels

- ▶ Metrics start an alert. Logs and traces explain it. Prediction logs connect a problem to the model.
- ▶ On a device, all four compete for the same small disk and the same link. This part is about how much to keep.

Source: After *Machine Learning Systems*, Volume II, chapter “ML Operations at Scale”, section “Observability architecture”.

A line for a person, a record for a program

Free text (110 bytes):

```
2026-10-03T14:00:00.000Z INFO live_detect: frame 0, 2 objects, 47.6 ms, 9.8 fps, conf 0.608, 60.9 C, 412.1 MB
```

Structured, one JSON object on each line (226 bytes):

```
{"ts":"2026-10-03T14:00:00.000Z","level":"INFO","device":"pi-07","app":"live_detect",  
  "model":"yolo11n_320_int8","event":"frame","frame":0,"latency_ms":47.6,"fps":9.8,  
  "objects":2,"conf_mean":0.608,"temp_c":60.9,"rss_mb":412.1}
```

- ▶ A program reads the second form with no pattern: one call of `json.loads` for each line. A dashboard selects a field by its name.
- ▶ The free text needs a regular expression, which breaks when a person changes the message.
- ▶ The cost is the size: the field names repeat in each record.

Source: Experiment of this part: the first record of the simulated hour (the listing breaks the JSON line in three).

Field	Content	Why
ts	Time in UTC, ISO 8601, with ms	Order of events, all devices on one time scale (Day 12)
level	DEBUG, INFO, WARNING, ERROR	Filter and count
device, app	Host name, program and its version	Which device, which release
model	Model file or version	Compare two versions
event	A fixed name: frame, slow_frame	Count events by name
Values	latency_ms, temp_c, rss_mb	The unit is part of the name

- ▶ Use the same names in all programs of the system: the log, the MQTT message, and the dashboard.
- ▶ Write the time in UTC. Convert to local time only on the screen.
- ▶ An event name is fixed text. The values go into fields, not into the name.

Level	Value	Meaning	Example
DEBUG	10	Detail for a developer	The boxes of each frame
INFO	20	Normal operation	Start, model loaded, summary
WARNING	30	Unusual, the program continues	A frame took 300 ms
ERROR	40	A task failed	The camera read failed
CRITICAL	50	The program cannot continue	The model file is missing

- ▶ The program writes only the records at or above its level. A device runs at INFO. Set the level from a setting, so that you can change it with no new code.
- ▶ A DEBUG call at the level INFO costs 0.55 microseconds: it can stay in the code.
- ▶ A good rule: an ERROR needs action by a person; a WARNING needs a look when it repeats.

Source: Level values of the Python module `logging`. Time: experiment of this part, x86 processor.

```
import json, logging, logging.handlers, time

class JsonFormatter(logging.Formatter):
    def format(self, record):
        data = {"ts": time.strftime("%Y-%m-%dT%H:%M:%SZ",
                                   time.gmtime(record.created)),
               "level": record.levelname, "event": record.getMessage()}
        data.update(getattr(record, "fields", {}))
        return json.dumps(data, separators=(",", ":"))

handler = logging.handlers.RotatingFileHandler(
    "detector.jsonl", maxBytes=1024 * 1024, backupCount=5)
handler.setFormatter(JsonFormatter())
log = logging.getLogger("live_detect")
log.addHandler(handler)
log.setLevel(logging.INFO)

log.info("frame", extra={"fields": {"frame": 7, "latency_ms": 47.6}})
log.debug("boxes", extra={"fields": {"boxes": [[12, 40, 200, 300]]}})
```

- ▶ Only the standard library. The DEBUG record is not written.
- ▶ The handler limits the files: the next frame explains it.

Call	Time
DEBUG at the level INFO	0.55 μ s
INFO to a JSON file	43.3 μ s
INFO through a queue	47.8 μ s

One record for each frame at 10 frames each second costs 0.4 ms in each second: less than 0.1 percent of the time.

- ▶ The file handler writes the record to the disk at each call. On a microSD card, one write can sometimes wait much longer than the mean.
- ▶ A QueueHandler puts the record into a queue. A second thread (QueueListener) writes it. On one core it saves no time, but a slow write no longer stops the frame loop.
- ▶ So the time of a log call is seldom the problem. The size of the logs is.

Source: Experiment of this part: median of 5 runs of 100 000 calls, one core of an x86 processor.

Format	Hour	gzip
JSON lines	7.89 MB	0.54 MB
CSV rows	3.57 MB	0.45 MB
SQLite, row a frame	2.03 MB	
SQLite, row a value	6.33 MB	

36 000 records: one for each frame at 10 frames each second, with 12 errors and 180 slow frames.

- ▶ JSON repeats the names, so it has about twice the size of CSV. gzip removes most of the repetition: a factor of 14.6.
- ▶ A real log compresses less: real values change more than the simulated ones.
- ▶ SQLite stores numbers in binary: 59.5 bytes for a row of 7 values. One row for each value (time, name, value) needs 30.9 bytes for each value.

Source: Experiment of this part: synthetic records; 1 MB = 1024 KB.

Rotation: a log with a size limit



when the file reaches 1 MB, each file moves one place

10 MB of records, limit 1 MB, 5 old files	Files	On the disk
Plain files	6	5.0 MB
Old files with gzip	6	0.31 MB

- ▶ The disk use has a fixed limit, whatever the program writes. The oldest records go first.
- ▶ `TimedRotatingFileHandler(when="midnight", backupCount=7)` keeps one file for each day, for 7 days.
- ▶ System logs: `logrotate`, and `SystemMaxUse=` in the settings of `journald`.

Source: Experiment of this part. The gzip step follows the Python logging cookbook (`namer`, `rotator`).

Sampling: keep fewer records

Rule (one hour)	Records	Size	Errors	Slow	Max. ms
Each frame	36 000	7.89 MB	12	180	398.1
1 in 10 frames	3600	807 KB	1	14	394.6
1 in 100 frames	360	80.8 KB	0	4	324.5
1 in 100, each WARNING and ERROR	548	124 KB	12	180	398.1
Summary each 60 s, each ERROR	72	18.9 KB	12	180	398.1

- ▶ “Slow”: the slow frames that the records show, of 180. “Max.”: the largest latency that the records show.
- ▶ A plain sample loses the rare events: 1 in 100 kept no error and 4 of 180 slow frames.
- ▶ Keep each rare event, and sample or summarize the normal ones. The textbook gives the same rule for a data centre: all errors, 1 percent of the successful requests.

Source: Experiment of this part: synthetic hour, seed 0. Rule: *Machine Learning Systems*, Volume I, chapter “ML Operations”.

```
{"ts":"2026-10-03T14:01:00.000Z","level":"INFO","device":"pi-07","app":"live_detect",  
  "model":"yolo11n_320_int8","event":"summary_60s","frames":600,"latency_mean_ms":47.1,  
  "latency_p95_ms":52.0,"latency_max_ms":198.4,"slow":1,"objects_mean":1.88,  
  "conf_mean":0.697,"temp_c":60.6,"rss_mb":411.9}
```

- ▶ One record of 292 bytes for 600 frames. It keeps what a person looks at: the mean, the p95, the maximum, and the count of slow frames.
- ▶ It loses the single frame. A slow frame or an error keeps its own record, so you can find its time.
- ▶ A repeated error (a camera that fails 10 times each second) gives one record and then a count each minute: “failed 600 times in 60 s”.
- ▶ The lab sends the same summary with MQTT to the dashboard.

Source: Experiment of this part: the first summary of the simulated hour.

```
edgeai_latency_ms{device="pi-07",stage="inference"} 47.6 at 14:00:00
edgeai_latency_ms{device="pi-07",stage="inference"} 46.9 at 14:00:15
edgeai_cpu_temp_celsius{device="pi-07"} 60.9 at 14:00:00
```

- ▶ A sample is a time, a name, some labels, and one number. A name with one set of labels is one **series**.
- ▶ The labels say where the value comes from: the device, the step, the model. A query selects or groups by label.
- ▶ Each new label value makes a new series. Do not put a frame number, an image name, or a person in a label: millions of series fill the storage.
- ▶ A log record can hold any field. A metric holds only numbers with a fixed set of labels, so it is small and fast to query.

Source: Notation of Prometheus; metric names of HW-08. Example labels and values.

Three ways to store metrics on a device

Store	Size of one frame	Good for	Limit
CSV file each day	104.7 bytes	A simple logger, a spreadsheet	A query reads the whole file
SQLite table	59.5 bytes	Queries by time, one file	One writer; commit in batches
Time-series database	1 to 2 bytes for each value	Many series, retention, a query language	One more service to run

- ▶ The companion book logs sensor readings to a CSV file each 60 s, and the bee counter writes one row each 10 s to SQLite.
- ▶ Prometheus is a time-series database: it compresses the samples and deletes samples older than its retention time (default 15 days).
- ▶ The lab uses the Python dashboard of HW-08 with CSV files. Option A of HW-08 uses Prometheus.

Source: Experiment of this part (CSV: the 13 fields of the record; SQLite: 7 numbers).
Prometheus documentation, "Storage".

```
import sqlite3, time

con = sqlite3.connect("metrics.db")
con.execute("PRAGMA journal_mode=WAL")
con.execute("PRAGMA synchronous=NORMAL")
con.execute("""CREATE TABLE IF NOT EXISTS frames (t REAL, latency_ms REAL,
            fps REAL, objects INTEGER, conf_mean REAL, temp_c REAL)""")
con.execute("CREATE INDEX IF NOT EXISTS frames_t ON frames (t)")

batch = []                                # rows of the last second
batch.append((time.time(), 47.6, 9.8, 2, 0.608, 60.9))
con.executemany("INSERT INTO frames VALUES (?,?,?,?,?,?)", batch)
con.commit()                             # one commit for each second, not each frame
rows = con.execute("SELECT avg(latency_ms), max(latency_ms) FROM frames "
                   "WHERE t > ?", (time.time() - 3600,)).fetchall()
```

- ▶ Day 12 measured one commit: 2.8 ms with the defaults, 0.027 ms with WAL and synchronous=NORMAL.
- ▶ The index on the time makes a query of the last hour fast. Delete old rows each night: DELETE ... WHERE t < ?.

Tier	Rows	Size
Each frame, 1 day	864 000	49.0 MB
Each second, 30 days	2 592 000	147.0 MB
Each minute, 365 days	525 600	29.8 MB
Total		225.9 MB
Each frame, 365 days		17 889 MB

A row of the SQLite table: 59.5 bytes. A summary row has about the same size.

- ▶ Recent data needs detail: you look for one slow frame of today. Old data needs only trends: the mean latency of last March.
- ▶ A job makes the summaries (downsampling) and deletes the detail when it is older than its tier.
- ▶ Prometheus: 20 series, each 15 s, for 15 days, is 1 728 000 samples: 1.6 to 3.3 MB at 1 to 2 bytes each.

Source: Calculation with the row size of the experiment; Prometheus formula of its documentation. 1 MB = 1024 KB.

- ▶ The Raspberry Pi stores its system and its logs on a microSD card. A full card stops the logs, the message queue of Day 12, and sometimes the system.
- ▶ Flash memory wears with each write. Many small writes (one write with a flush for each frame) wear it more than one large write each second.
- ▶ The rules of this part protect the card: a size limit (rotation), fewer records (sampling, summaries), and writes in batches.
- ▶ Check the space: `df -h /`, `du -sh logs`, `journalctl --disk-usage`.
- ▶ An option: write the detailed log to a RAM disk (`/run` or a `tmpfs`) and keep only the summaries on the card. A restart deletes the detail.

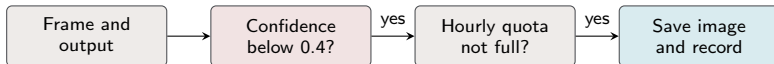
Key point: Give each log a limit in bytes. A log with no limit fills the disk on a day when nobody watches.

Do not log	Risk	Log in its place
Images, audio	Faces, voices, homes	Counts, classes, scores
Text of a user (Day 10)	Names, health, opinions	Length, intent class
Passwords, keys	Access to the system	Nothing; a key ID
Exact place with time	The habits of a person	A zone, a coarse time

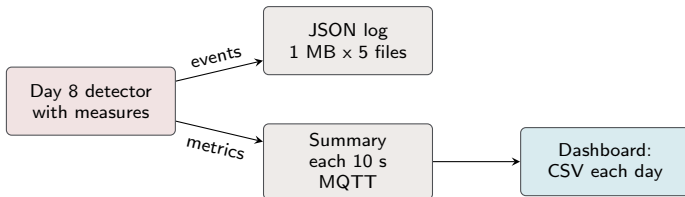
- ▶ **Data minimization:** log only what the purpose needs. A person counter needs a count, not the image of each person.
- ▶ **Purpose and retention:** write why you keep each field and for how long. Rotation enforces the time.
- ▶ **Access:** a log file on the device is readable by its users. Set the file rights; do not send logs over an open link.
- ▶ An edge device can keep the raw data local. A log that copies it to a server removes this advantage.

Source: *Machine Learning Systems*, Volume I, chapter “Responsible Engineering”; Volume II, chapter “Edge Intelligence” (GDPR).

- ▶ Part 3 needs examples of the inputs when the model is not sure: they become the new training data.
- ▶ Keep a small sample: for example, the frames with a confidence below a threshold, at most N each hour.
- ▶ Store them in a separate folder with its own size limit and a short retention. Do not mix them with the normal log.
- ▶ Images of people need a notice in the room, or a blur of the faces on the device before the image is stored.
- ▶ Record with each sample: the time, the model version, the output, and the reason for the sample.



Source: The threshold 0.4 and the quota are example values.



- ▶ Part A: the detector writes a structured JSON log with rotation: the start, each WARNING and ERROR, and a summary.
- ▶ Part B: the same summary goes as an MQTT message to the topic `edgeai/gNN/pi/metrics`. The dashboard of HW-08 stores it in a CSV file for each day and draws it.
- ▶ The lab keeps no image. The drift check of Part C uses the confidence and the input statistics.

The detector of the lab writes JSON records of 230 bytes. Calculate the storage for one day ($1 \text{ MB} = 1024 \text{ KB}$) for each rate:

- A. one record for each frame, at 10 frames each second;
- B. one record each second;
- C. one summary each minute.

The device has a log budget of 1 GB for 30 days. Which rates fit? Which fit when gzip compresses the old files by a factor of 14.6?

Answer: Storage of one day of logs

Rate	Records	One day	30 days	30 days, gzip
A. Each frame	864 000	189.5 MB	5685 MB	389.4 MB
B. Each second	86 400	19.0 MB	568.5 MB	38.9 MB
C. Each minute	1440	0.32 MB	9.5 MB	0.65 MB

- ▶ One day: $864\,000 \times 230 = 198\,720\,000$ bytes = 189.5 MB.
- ▶ Without gzip, B and C fit 1 GB (1024 MB) for 30 days. With gzip, A also fits.
- ▶ The factor 14.6 came from synthetic records. Measure the factor on real logs before you plan with it.

Part 1 Failures of deployed edge AI

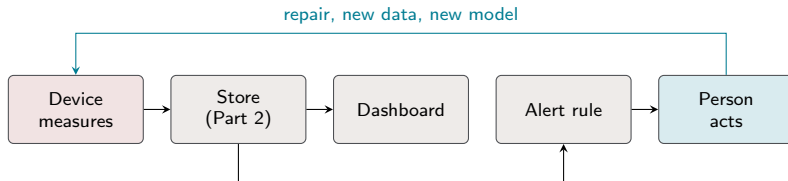
Exercise: for three failures, name the metric that shows the failure first

Part 2 Logging

Exercise: calculate the storage that one day of logs needs at three sampling rates

Part 3 Visualization and alerts

Exercise: set alert thresholds from a given metric history



- ▶ A dashboard shows the metrics to a person who looks. An alert calls a person who does not look.
- ▶ Nobody watches a dashboard of 30 devices all day. The alert rules do the watching; the dashboard helps to find the cause.
- ▶ An alert is useful only when a person can act on it: clean a lens, restart a program, collect new data.

Key point: Each alert needs an owner and an action. An alert with no action teaches people to ignore alerts.

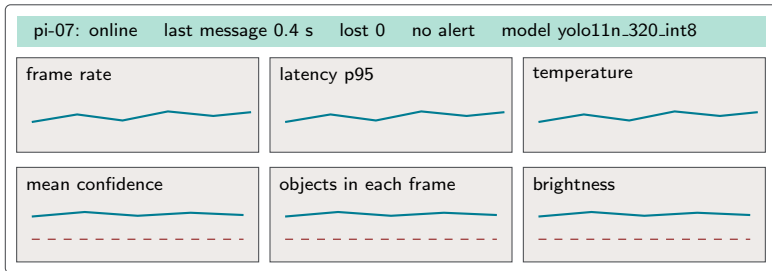
One dashboard for each question

View	Question	Panels
All devices	Which device has a problem now?	One row for each device: state, last message, active alerts
One device	Is this device healthy?	System and model metrics of the last hours, with the reference band
Investigation	Why did it change, and when?	Several metrics on one time axis, the log events, the model version

- ▶ Start with the question, then select the panels. A panel that answers no question takes space and attention.
- ▶ The lab builds the second view for one Raspberry Pi. The first view needs the metrics of all devices in one place.

Source: After *Machine Learning Systems*, Volume II, chapter “ML Operations at Scale”, section “Dashboard design”.

The layout of a device dashboard



- ▶ Top: one line of state, which a person reads in one second. Its colour changes with an alert.
- ▶ One row for the system, one row for the model. All panels share the same time axis, so one event lines up in all panels.
- ▶ Each panel shows its unit, its threshold, and the normal band.

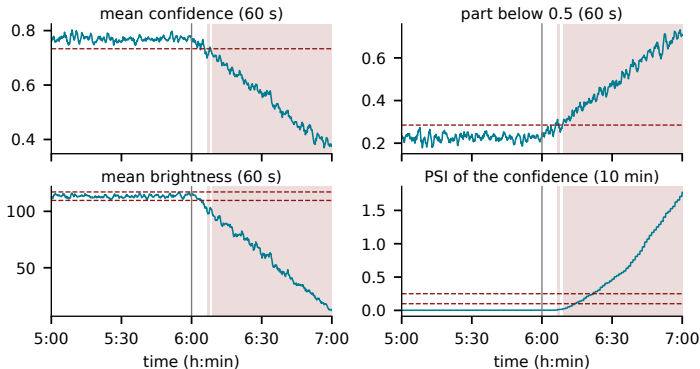
Source: Example layout. The Python dashboard of HW-08 has a state bar and six charts.

Panel	Use	Example
Time series	A value over time, its trend, its noise	Latency p95 of the last 6 hours
Single value	The current state, with a colour	Temperature now: 61 °C
Table	Events and their details	The last 10 alerts
Histogram	The distribution of a value	Confidence of the last hour against the reference

- ▶ A mean hides the bad cases. Show the p95 or the maximum next to the mean (Day 9).
- ▶ A change of a distribution can leave the mean the same. A histogram against the reference shows it.
- ▶ Use a fixed range of the axis for a value with known limits (a confidence from 0 to 1), so that a change is visible.

Source: The temperature is an example value.

A dashboard during a drift

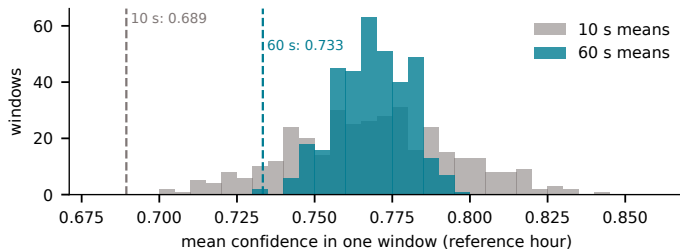


- From 6:00 the room becomes darker. After one hour, each frame has one tenth of the light.
- Red area: the alert "mean confidence of 60 s below the threshold for 60 s". It starts after 390 s.
- The brightness shows the cause. The PSI changes slowly.

Source: Simulation of this part (values of Part 1).

Metric	Limit	From
Temperature	80 °C for 30 s	Throttling starts at 80 °C (Part 1)
Last message	Older than 15 s	Keep-alive and the last will (Day 12)
Free disk	Below 10 percent	The disk must not fill (Part 2)
Queue age	Oldest message older than 5 min	Store and forward (Day 12)
Frame rate	Below the need of the task	The requirement of the application

- ▶ Some limits come from the hardware, the protocol, or the task. They need no history: the device fails at the limit.
- ▶ The first two rows are rules of HW-08 for the lab (HighTemperature, DeviceSilent). The other limits are example values.
- ▶ A model metric has no such limit. Its normal value depends on the scene, so it comes from a history.



- ▶ Record a reference period with no problem (here one hour). Take the mean minus 3 standard deviations of the windows.
- ▶ 10 s means: 0.769 ± 0.026 , threshold 0.689. 60 s means: 0.769 ± 0.012 , threshold 0.733.
- ▶ A longer window has less noise, so its threshold can be closer to the normal value. It sees a smaller drift, but later.
- ▶ When the values are not normal, use a percentile (for example the 0.1 percent point).

Source: Simulation of this part (reference hour).

How many false alerts?

$$P(\text{at least one false alert}) = 1 - (1 - \alpha)^N$$

Rule: mean – 3 sd, one day	Checks	False alerts	At least one
Check each 10 s	8640	11.7	100 %
Check each minute	1440	1.9	86 %
20 metrics, each minute	28 800	38.9	

- ▶ α : the chance that one check of a normal value passes the threshold (0.00135 for 3 sd on one side). N : the number of checks.
- ▶ Many checks make false alerts certain. After some days of false alerts, people do not read them any more.
- ▶ Fewer, better rules: a longer window, a duration, and a rule for each cause, not for each metric.

Source: Formula: *Machine Learning Systems*, Volume II, chapter “ML Operations at Scale” (alert fatigue). Values: normal distribution, independent checks.

Setting	Effect	Cost
Duration (for)	Fire only when the condition stays for a time	Adds this time to the delay
Hysteresis	Clear only when the value is back past a second limit	The alert stays longer
Severity	Warning (look today) or critical (act now)	Two thresholds to keep

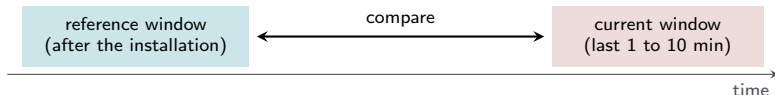
- ▶ The same ideas as the decision layer of Day 12: one noisy value must not switch the state.
- ▶ An alert message names the device, the metric, its value, the threshold, the window, and the model version. The person then knows where to start.
- ▶ Send the alert where a person sees it: an MQTT topic, an e-mail, a message, or the LED of a device.

Source: The content of an alert message: *Machine Learning Systems*, Volume I, chapter “ML Operations”.

```
- alert: LowConfidence
  expr: avg_over_time(edgeai_confidence[30s]) < 0.6
  for: 10s
  labels:
    severity: warning
  annotations:
    summary: "Low confidence on {{ $labels.device }} of {{ $labels.group }}"
```

- ▶ Option A of HW-08 (Prometheus): the mean confidence of 30 s below 0.6, for 10 s. A second rule fires when a device sends nothing for 15 s.
- ▶ Option B (the Python dashboard of the lab): the mean confidence of the last 20 messages below 0.6. The state bar becomes red.
- ▶ The value 0.6 is a first guess of HW-08. In the lab you replace it with a value from your own reference period.

Source: Labs/hardware/HW-08/grafana/alert_rules.yml and python_dashboard/dashboard.py of this course.



Signal	Test	It sees
Confidence	Mean, or the part below 0.5	Any change that harms the model
Input statistics	Brightness, sharpness against a band	The cause: light, focus, noise
Distribution	PSI or KS of a value against the reference	A change of shape, also with the same mean

Each test compares a current window with a reference window. The labels come later, if they come.

$$\text{PSI} = \sum_{i=1}^n (a_i - e_i) \ln \frac{a_i}{e_i}$$

- ▶ Divide the range of the value into n fixed bins (here: 10 bins of the confidence, 0 to 1). e_i : the part of the reference in bin i . a_i : the part of the current window.
- ▶ A bin with no value makes the logarithm infinite. Give each empty bin a small part (here 0.0001).
- ▶ Common limits: below 0.1 stable, 0.1 to 0.25 a warning, above 0.25 a large change. They are conventions, not laws.
- ▶ In the simulation, the PSI of 10 min against the reference hour stayed below 0.017 in the normal hours, also with the short dark event. After one hour of the gradual drift it was 1.77.

Source: Formula and limits: *Machine Learning Systems*, Volume I, chapter "ML Operations".
Values: simulation of this part.

Rule	False	Event	x 0.5	x 0.25	Gradual	Blur
Mean conf. 10 s	4	yes	missed	10 s	350 s	10 s
Mean conf. 10 s, for 60 s	0	no	missed	60 s	940 s	60 s
Mean conf. 60 s	1	yes	40 s	20 s	230 s	20 s
Mean conf. 60 s, for 60 s	1	yes	90 s	70 s	390 s	70 s
Part below 0.5, 60 s	3	yes	missed	20 s	350 s	10 s
Brightness, 60 s	3	yes	10 s	10 s	250 s	missed
PSI above 0.25, 10 min	0	no	missed	480 s	1320 s	540 s

- ▶ “False”: alerts in 5 normal hours (with the short event). “Event”: the dark event of 30 s started an alert.
- ▶ A delay counts from the start of the drift. “Missed”: the alert was active for less than half of the last 30 minutes.

Source: Simulation of this part: thresholds mean ± 3 sd of the reference hour; checks each 10 s.

- ▶ **A short window is fast and noisy:** 4 false alerts in 5 hours, and it misses the small drift (light $\times 0.5$).
- ▶ **A duration removes the false alerts,** also the short event, but it adds delay and misses the small drift.
- ▶ **A longer window** sees the small drift in 40 s with 1 false alert: the short event, which is a real change of 30 s.
- ▶ **The brightness** sees each change of the light in 10 s, and it does not see the blur. It names the cause.
- ▶ **The PSI** gave no false alert. It needed 8 to 22 minutes and missed the small drift: a check for a daily report.

Key point: Alert on one stable model rule (a longer window with a duration). Show the input statistics next to it to find the cause.

Source: Simulation of this part. The frames are independent; a real camera gives correlated frames and more noise between windows.

Drift hour	Accuracy
No drift	77.2 %
Light $\times 0.5$	72.7 %
Light $\times 0.25$	66.7 %
Gradual to $\times 0.1$	57.5 %
Blur of 2 pixels	59.4 %

The accuracy of the frames in the hour after the start of the drift.
Nobody on the device can measure it.

- ▶ The small drift costs 4.6 points, and four of the seven rules missed it. The large changes are easy to see.
- ▶ The alert does not repair the accuracy. It shortens the time with wrong answers, and it starts the work that repairs it.

Source: Simulation of this part: the stored result of each image.

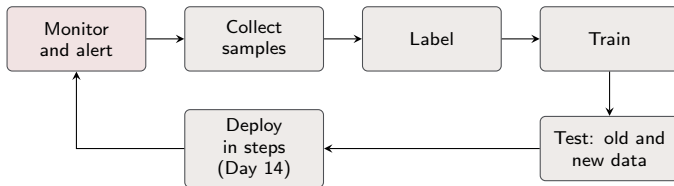
Diagnose

1. Look at the device: camera, light, lens, position.
2. System metrics: temperature, memory, restarts.
3. Input statistics: which one changed?
4. Changes: a new model, a new program, a new place.
5. Compare with the same hour of an earlier day.

Incident report

- ▶ Time of the start, of the alert, of the end
- ▶ How you found it: rule, value, threshold
- ▶ Effect: what was wrong, for how long
- ▶ Cause and action
- ▶ What changes so that it does not happen again

The lab hands in a short incident report for the drift event of Part C. The report is a record of what happened, not a search for a person to blame.



- ▶ The samples come from the prediction log of Part 2: frames with a low confidence from the time of the drift.
- ▶ Test the new model on the old test set and on new data. A model that learns the dark scene must not forget the bright one.
- ▶ Retrain on a schedule, or when a drift rule fires. A trigger acts sooner after a sudden change; a schedule is simple to plan.

Source: Schedule and trigger: *Machine Learning Systems*, Volume I, chapter “ML Operations”.

Not every alert needs a new model

Cause	Signal	Action
Sensor fault	Failed input checks; a stuck or black frame	Repair or clean; no training
Training-serving skew	Low confidence at once after a release	Fix the code; roll back
Input drift ($P(x)$)	Input statistics and confidence change	New data of the new scene; retrain
Prior drift ($P(y)$)	Class counts change, confidence stays	Often nothing; check the thresholds
Concept drift ($P(y x)$)	Only labels show it	New labels; retrain
Heat, leak	System metrics	Cooling, a fix of the code

- ▶ Find which part changed before you retrain. A model trained on the data of a broken sensor learns the fault.
- ▶ The first two rows are the most frequent and the cheapest to fix.

Source: *Machine Learning Systems*, Volume I, chapter “ML Operations” (diagnose the drift before retraining).

Part	Work	From this lecture
B Dashboard	Summaries with MQTT, the Python dashboard of HW-08, a reference period	Device view, panel types
C Drift	Change the light or show a new object; find the event from the confidence	Windows, input statistics
D Alert	Set a threshold from your reference; fire and clear the alert	Mean — 3 sd, duration

- ▶ Record the reference period before you cause the drift. Without it, no threshold has a meaning.
- ▶ The deliverable: the dashboard and a short incident report of the drift event.

A reference hour of the stream of this part gives these 60 s means of the confidence, one each 5 minutes:

0.782, 0.782, 0.764, 0.780, 0.762, 0.771, 0.764, 0.787, 0.780, 0.775, 0.767,
0.770

- A. Calculate the mean and the standard deviation. Set a threshold at the mean minus 3 standard deviations.
- B. All 355 windows of the same hour give 0.769 ± 0.012 . Which threshold is better, and why?
- C. The Raspberry Pi throttles at 80°C . Set a warning and a critical rule for the temperature, each with a duration.

- A. Mean 0.774, standard deviation 0.0085, threshold $0.774 - 3 \times 0.0085 = 0.748$.
- B. 12 values are too few: they give a small standard deviation by chance. 5 percent of the 355 windows of the reference hour are below 0.748: with one check each minute, about 73 checks each day fire with no problem. The threshold of all windows is $0.769 - 3 \times 0.012 = 0.733$. Use a long history, and check how often the threshold fires in it.
- C. The temperature has a limit from the hardware, so it needs no history. For example: a warning above 75°C for 5 minutes (the cooler is at full speed); a critical alert above 80°C for 30 s (throttling, the rule of HW-08).

- ▶ A deployed model can lose accuracy with no error: data drift, sensor faults, heat, memory leaks. System metrics show the state of the device. Model metrics show the state of the predictions.
- ▶ A structured log has one record with fixed fields for each event. Log levels, rotation, and sampling keep the logs inside the storage. A log keeps no personal data that the task does not need.
- ▶ A dashboard shows the metrics over time. An alert fires when a metric passes a threshold for a minimum time, not for one noisy value.
- ▶ With no labels, a drop of the confidence or a change of the input statistics shows drift. Then the loop starts again: collect, label, train, test, deploy.

Key point: Measure the device and the model, keep the logs small, and let an alert tell you about drift before the users see it.

Book chapters

- ▶ *Machine Learning Systems* (mlsysbook.ai), Volume I, chapter “ML Operations”, section “Production Operations”.
- ▶ *Machine Learning Systems*, Volume II, chapter “ML Operations at Scale”, section “Monitoring at Scale”.
- ▶ *Edge AI Engineering*: “Experimenting with SLMs for IoT Control”, section “Adding Data Logging”.

Papers

- ▶ “A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks”, Hendrycks and Gimpel, 2017.
- ▶ “Failing Loudly: An Empirical Study of Methods for Detecting Dataset Shift”, Rabanser, Günnemann, and Lipton, 2019.

Tools

- ▶ Prometheus: <https://prometheus.io/docs/>
- ▶ Grafana: <https://grafana.com/docs/grafana/latest/>

This lecture adapts material from these sources:

- ▶ *Machine Learning Systems* by Vijay Janapa Reddi and contributors, Harvard University. mlsysbook.ai, CC BY-NC-SA 4.0.
- ▶ *Edge AI Engineering: Raspberry Pi* by Marcelo Rovai. github.com/Mjrovai/EdgeML_Made_Ease_ebook and github.com/Mjrovai/EdgeML-with-Raspberry-Pi, GPL-3.0.

Material from a source with the licence CC BY-NC-SA 4.0 keeps that licence.